

Tellak - Hamam Brawler

Technical Design Document

Course: BMI4242 Game Programming

Department: Computer Engineering

Academic Year: 2025/2026 Spring

Instructor: Asst. Prof. Dr. Alpay DORUK

Team Members and Roles

- **Burak Saran (2211504067)** : Establishing the core loop architecture and optimizing the `runArena()` logic. Managing action and damage synchronization; configuring animation phases and FX (effects) trigger controls within `anim.js`. Strengthening the core combat flow, specifically focusing on `processActionHit()`, `damageEnemy()`, and `hitApplied` functions to minimize bugs.
 - **Adem Buran (2211504033)** : Designing the advanced pathfinding system and developing the `findNavPath`, `getEnemyPath`, and `pickEnemyChaseStep` functions. Implementing enemy spawn controls, handling stuck situations (*stuck handling*), and creating navigation *fallback* mechanisms to prevent system crashes. Executing enemy DOM updates, managing layer depth (`zIndex`), and optimizing the rendering order using `transform` synchronization.
 - **Hatice Serin (2211504054)** : Preparing the project technical documentation and keeping the `README.md` file up to date. Improving the overall game feel and conducting quality controls for animation synchronization. Performing the final tests and overseeing the ultimate gameplay inspection before delivery.
-

1. Game Overview

Elevator Pitch

Tellak — Hamam Brawler is a dynamic 2D top-down beat 'em up action game built with modern web technologies. Set in a traditional Turkish bath atmosphere, players knock out magandas using Ottoman slaps and kicks, and recover their health by drinking Turkey's national beverage, **sodali ayran**, when their energy is low.

Core Mechanics

- **8-Way Movement:** The player moves freely across the bathhouse map using the W, A, S, and D keys. The character's facing direction updates instantly based on either the mouse position or the movement vector.
- **Attack Combos and Abilities:**
 - **Punch (J):** Deals 1 damage point and has a 1-second cooldown.
 - **Kick (K):** Deals 2 damage points and has a 2-second cooldown.
 - **Ottoman Slap (L):** Deals Area of Effect (AoE) damage, inflicting 3 damage points on all enemies within range. It has a 10-second cooldown and features a **25% chance for an instant kill (Instakill)**.
- **Health Regeneration (Sodali Ayran - B):** When the player's health (HP) is low, drinking "sodali ayran" instantly refills the health bar completely. To prevent abuse of this mechanic, it has a 30-second cooldown.
- **Difficulty Scaling (Dynamic Spawning):** As the game progresses, the enemy spawn rate and wave intensity increase. As the player clears out enemies, their score (Kill Count) and total damage dealt are updated in real-time on the HUD.

Win/Loss Conditions

- **Win Condition:** The game features an endless survival mode. The player's objective is to survive in the bathhouse for as long as possible and achieve the highest score by neutralizing the maximum number of magandas.
 - **Loss Condition:** When the player's health falls to zero ($\text{PLAYER_MAX_HP} = 10$), the game pauses, the endofgame sound effect triggers, and a "Game Over" panel appears on the screen.
-

2. Software Architecture

High-Level Structure

The game is developed using a **Custom 2D HTML5 Canvas Game Engine** built on top of the **React (v18)** architecture, a modern web stack composition. To avoid the overhead of the Virtual DOM and ensure a smooth gameplay experience (60 FPS), the core engine and render loop are fully encapsulated using **React Hooks (useRef)** and optimized pure JavaScript functions.

- **UI Layer (React State):** Manages only the low-frequency interface elements that require infrequent updates, such as menu transitions, audio settings, and the HUD (Health bar, Cooldown indicators, Score).

- **Game Loop Layer (Direct DOM & Canvas Render):** Character, effect, and enemy logic states are updated in memory and rendered directly onto the Canvas using requestAnimationFrame or high-resolution, time-based performance.now() loops.

Core Managers / Controllers

The game architecture is separated into the following subsystems via modular reference objects:

1. **Input Controller:** Asynchronously listens to keyboard inputs (WASD + JKL) by maintaining a Set data structure named arena.current.keys. This prevents input loss during simultaneous key presses (e.g., throwing a punch while running diagonally).
2. **Animation Manager (window.TELLAK):** Located in the anim.js file, this module manages sprite-based animations using a **Mathematical Procedural Animation** engine instead of static frame-by-frame sheets.
3. **Audio Manager:** Asynchronously loads audio effects (sounds.current) from categorized folder structures. It dynamically controls audio files for background music (background), dealing damage (damagegiven), taking damage (damagetaken), drinking ayran (drinkingayran), game over (endofgame), and enemy death (enemydeath) using a linear volume equalization (syncAudioVolumes) algorithm.

Design Patterns

- **Singleton Pattern / Namespace Isolation:** All animation and auxiliary mathematical libraries are wrapped inside an IIFE (Immediately Invoked Function Expression) as (function () { ... })() and attached to the global window.TELLAK object. This prevents global scope pollution.
- **State Pattern (State Machine):** The instantaneous states of characters and enemies (Running, Striking, Drinking Ayran, Taking Damage, Dying) are managed through state variables. For instance, while the player is in the drink state, movement inputs are locked.
- **Data-Driven Architecture:** Direction vectors and animation curves are maintained in static configuration objects (DIRS, pose). Instead of using hardcoded values within the code, these data tables are queried.

3. Algorithmic Implementation & Data Structures

Data Structures

- **Navigation Grid (2D Flattened Grid Array):** To store the walkable areas of the bathhouse map, the navGrid.current structure utilizes an optimized **Uint8Array** composed of contiguous memory blocks instead of a traditional 2D matrix. Cell coordinates (x,y) are mapped to a unique index using the following formula:

$$\text{index} = y \times w + x$$

- **BFS Search Queue (Typed Array Queue):** In the AI pathfinding algorithm, a high-performance **Queue** data structure is implemented using a fixed-size **Int32Array** and two pointers (head, tail). This completely eliminates the $O(N)$ re-indexing overhead caused by JavaScript's built-in Array.push/shift functions.
- **Enemy Pool (Linear Dynamic Array):** All active enemy objects are stored in the enemiesRef.current array. This array is scanned during every frame to filter out and garbage-collect dead enemies from memory.

Algorithms (Algorithms and Mathematical Models)

1. Canvas-Based Spatial Masking

During the game loading phase, a mask image named movinganddrinkingspaces.jpg is drawn onto an offscreen canvas. The walkable (ZONE_GREEN) and restricted areas (ZONE_PINK) of the bathhouse are scanned using the RGB color distance formula (Manhattan Distance):

$$\text{Distance} = |R_{\text{pixel}} - R_{\text{zone}}| + |G_{\text{pixel}} - G_{\text{zone}}| + |B_{\text{pixel}} - B_{\text{zone}}|$$

Pixels falling below the specified tolerance threshold ($\text{ZONE_TOL} = 70$) are written into zoneMask.current.cells as 1 or 2. Consequently, whenever the map design changes, physical boundaries can be updated simply by modifying the mask image without rewriting any code.

2. AI Breadth-First Search (BFS) Pathfinding

To enable enemies (magandas) to intelligently pursue the player inside the bathhouse, a **Breadth-First Search (BFS)** algorithm executes within the findNavPath function.

- The map is partitioned into grid cells with a size of $\text{NAV_CELL_SIZE} = 18$ pixels.
- Starting from the enemy's current cell, the algorithm scans the 4-directional neighborhood matrix ($[1, 0]$, $[-1, 0]$, $[0, 1]$, $[0, -1]$).
- The parent node histories of the visited cells are tracked in a parents array. Once the target (the player) is reached, the path is retraced (reversed) to generate an optimized route.

3. Mathematical and Functional Procedural Animation (Procedural Easing)

To prevent the high memory consumption associated with traditional sprite sheets, animations are calculated mathematically based on a time-dependent phase parameter ($p \in [0,1]$):

- **Linear Interpolation (Lerp):** Utilized for character position and scale transitions:

$$\text{lerp}(a,b,t)=a+(b-a)*t$$

- **Cubic Easing Algorithms:** To deliver an organic sense of acceleration (the tension before a strike and the sudden impact) during punches, kicks, or Ottoman slaps, easeIn, easeOut, and easeInOut cubic functions are applied:

$$\text{easeIn}(t)=t^3$$

$$\text{easeOut}(t)=1 - (1 - t)^3$$

4. Optimization & Memory Management

Performance Bottlenecks & Solutions

- **HTML5 Canvas Redraw Optimization:** Instead of recalculating the entire map's pixels from scratch during every render frame, the background image is cached in the browser memory (preload sprites). Only the dynamic objects are rendered using GPU-accelerated transformation matrices (ctx.translate, ctx.rotate, ctx.scale) based on their calculated bounding box areas.
- **Garbage Collection Mitigation:** Continuously creating new objects ({}, []) within the game loop triggers the JavaScript Garbage Collector, leading to micro-stuttering. In this project, the parents and queue structures utilized in the BFS algorithm are allocated only once as an **Int32Array** at game initialization. Instead of being recreated within the loop, these structures are optimized using an overwrite approach.

Memory Management

- **Smart Asset Pipeline:** All visual assets used in the game (tellak_front, tellak_back, enemy_front, enemy_back, map) were uniquely generated using AI integration to fit the bathhouse theme and injected into memory without compromising visual quality.
- **Audio Asset Preloading:** Audio files are preloaded into memory all at once during the initial game loading phase; they are not dynamically cleared or re-uploaded during gameplay, thereby preventing disk I/O blocking.

5. Version Control & Workflow

GitHub

This project is currently a fully playable version. All core gameplay mechanics, enemy AI, animation synchronization, and audio systems are completely functional. The link for the live demo is provided below:

<https://sorcery0358.github.io/tellak-hamam-brawler/>

Integration Challenges & Solutions

- **Animation Synchronization:** Due to the asynchronous nature of the animation system, precisely capturing the exact moment of impact (for example, the precise instant the Ottoman Slap connects at phase $p=0.45$) proved difficult, causing the damage registration to fall out of sync with the visual animation. Damage was either being processed too early while the animation was still in its wind-up phase, or too late after it had already passed the recovery phase. This misalignment created a weak and inconsistent game feel.
 - **Solution:** A phase-based flag mechanism was integrated into the requestAnimationFrame loop. A specific critical timestamp (`hitTime = 0.45`) was defined for each individual action (punch, kick, slap). Inside the `runArena` function, when the animation phase reaches this threshold, the `hitApplied` flag is verified to ensure that damage is inflicted exactly once. Simultaneously, the `processActionHit` function executes the following logic:
 - Scans the enemy array (`enemiesRef.current`).
 - Validates the strike range by performing a distance check ($d \leq \text{effRange}$).
 - Triggers the `damageEnemy()` function on the correct frame (inflicting distinct damage points based on the strike type: Punch = 1, Kick = 2, Slap = 3).

This architecture synchronizes the animation and damage mechanics through a data-driven event flag system, making it resilient to frame rate variations. Consequently, visual consistency and game feel have been seamlessly optimized.

- **React State Closure Issue (Stale Closure):** The `requestAnimationFrame` loop requires continuous access to React state values (such as music volume, SFX volume, and overall game state); however, due to JavaScript closure mechanics, these state values often became stale inside the loop. For instance, when the `musicVol` state was updated, the already-running `frame()` function would continue to read and apply the outdated value.
 - **Solution:** All critical state variables (`musicVol`, `sfxVol`, `resumingCount`) were mirrored using `useRef`. Simultaneously, a `useEffect` hook was implemented to synchronize these ref values upon every state update (e.g., `useEffect(() => { musicVolRef.current = musicVol; }, [musicVol])`). This architectural pattern guarantees that the `runArena()` loop always evaluates the most up-to-date values in real-time.
- **Enemy Pathfinding Squeeze (Enemy Stuck in Corners):** The BFS algorithm occasionally caused enemies (magandas) to get trapped in tight corners of the maze-like bathhouse map. Even though recalculations were triggered when an enemy reached the same coordinate, the algorithm failed to find alternative paths, leaving the enemy immobilized.

- **Solution:** A two-tier fallback mechanism was deployed. First, the `getEnemyPath()` function caches pathfinding results for a duration of 350ms. If an enemy remains stationary, a `stuckFrames` counter increments, applying a pathing penalty (`stuckPenalty`) that forces the algorithm to prioritize alternative routes. As a final resort, the `pickEnemyChaseStep()` function evaluates up to 15 different directional angle offsets to find an alternate escape route (`angleOffsets = [0, 18, -18, 36, -36, ...]`).
- **Pause/Resume State Synchronization:** Managing state transitions when the player toggled the Pause menu introduced complex synchronization issues. During the resume sequence, a countdown (`resumingCount`) would trigger, yet enemies would occasionally continue moving or audio components failed to pause correctly.
 - **Solution:** An explicit state machine was established, synchronizing the `resumingCountRef` with the `resumingCount` `useState` hook. When a pause request is received while a countdown is already active, it is immediately aborted via `cancelResumeCountdown()`; otherwise, a new countdown is initiated. A `useEffect` hook monitors the `[gameStarted, paused, resumingCount]` dependencies to ensure all audio sources are instantly paused or resumed in perfect sync.
- **Gamepad Input Bounce Debouncing:** Gamepad inputs—specifically the Start button and D-pad directional keys—frequently triggered rapid, unintended double-registrations (the bounce problem). This caused the menu navigation to skip target selections or caused the game to abruptly pause and unpause.
 - **Solution:** A **220ms debounce** threshold was implemented for all pause toggles utilizing a `lastPauseToggleRef`. Any incoming toggle request is processed only if at least 220ms have elapsed since the prior toggle; otherwise, it is discarded. Similarly, for menu navigation (up/down), a `prevDpad` state is maintained to process inputs strictly on an **edge trigger** basis: `upEdge = upBtn && !prevDpad.up`.
- **DOM Updates vs. Game Loop Timing:** The rendering of enemy DOM elements (`el.style.transform`) occasionally fell out of sync with the underlying physics and animation coordinate updates. If `applyPoseToEnemy()` failed to execute a position update during a frame, the obsolete transform calculation remained visible on screen.
 - **Solution:** The system now guarantees that either `applyPoseToEnemy()` or a direct `el.style.transform` update is enforced during every single frame. If an enemy is not actively performing an action (i.e., in an idle or walk state), the positional coordinates are explicitly forced via a fallback string: `en.el.style.transform = en.el.style.transform || `translate3d(en.xpx, ${en.y}px, 0)``. This safety fallback ensures that moving entities never suffer from visual positioning discrepancies.